



Session 3:

Build Your First PoC (Part-2)

03 **FINISH, DOCUMENT, STABILISE**

Documentation-First Development

Prompt Claude Code to write the README as you build — not after. A PoC without a README is not handoff-ready.



Forces scope clarity

Writing what it does exposes ambiguity. If you cannot describe it clearly, it is not done.



Makes handoff instant

Anyone picking this up — engineering, a colleague, future you — can start without a briefing.



Confirms the build

Generating the README reveals gaps between what you intended and what Claude actually built.

THE README PROMPT

```
"Write a README.md for this project. Include:  
- What it does (1 paragraph)  
- Who uses it and when  
- How to install and run it  
- What a successful run looks like  
- What is out of scope"
```

Run this now — before you finish building. Edit what is wrong. Keep it current.

The Self-Review Prompt

Before you call it done — ask Claude to stress-test it. 30 seconds. Saves your demo.

THE PROMPT

```
"What are the top 3 things most likely to break or fail in a live demo of this PoC? Be specific."
```

Classify each risk Claude identifies:

Blocking

Breaks the demo entirely

→ Fix it now — before the commit

Deferrable

Edge case unlikely in a controlled demo

→ Note it in the README. Move on.

Known Limitation

Out of scope by design

→ Confirm it is in your 'out of scope' list.

Saving Your Work — Git Commit

Commit before Session 3. Claude generates the message. You review and push.

Generate the commit message

1

```
$ claude "Write a Git commit message for this session. Include what it does, what it does not do, and any known limitations."
```

Review it. Edit if vague.

Stage your files

2

```
$ git add .
```

Check git status first. Only stage files you intended to change.

Commit with the generated message

3

```
$ git commit -m "[CLAUDE-GENERATED MESSAGE]"
```

This PoC runs through Sessions 3 and 4. A clear message matters.

Live Build Part 2 — Complete, Document, Commit

HANDS-ON LAB

Finish the PoC. Make it explainable. Make it committable.

1

Complete the working core

If Part 1 left anything unfinished — finish it. One function, one clean output.

2

Run the self-review prompt

Fix anything blocking. Note what is deferrable in the README.

```
$ claude "Top 3 things most likely to fail in a live demo?"
```

3

Generate and review your README

Run the README prompt. Fix any inaccuracies before saving the file.

```
$ claude "Write a README.md for this project..."
```

4

Add basic error handling

Apply the three error handling prompts to your most likely failure points.

5

Commit to Git

Generate commit message · Stage files · Commit · You are done for Session 2.

```
$ git add . && git commit -m "[message]"
```

✓ **SUCCESS: PoC runs · Self-review done · README saved · Error handling added · Committed to Git**

What You Now Have

SCOPED & BRIEFED

- ✓ PoC brief: one journey, one integration, one testable outcome
- ✓ Scope test passed
- ✓ CLAUDE.md created and verified

BUILT & RUNNING

- ✓ Working PoC — core logic runs
- ✓ Build loop applied: prompt → run → fix → repeat
- ✓ Self-review prompt run — blocking risks addressed

DOCUMENTED & COMMITTED

- ✓ README generated and reviewed
- ✓ Basic error handling added
- ✓ Committed to Git with a descriptive message

This PoC continues in Sessions 3 and 4. Keep it open — you will build on it.

Q&A

